DATA STRUCTURE FUNDAMENTALS

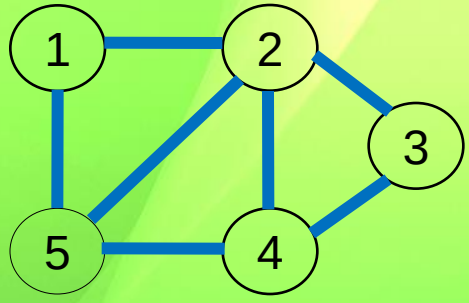
Prof. Dr. Md. Rafiqul Islam

CHAPTER 8

GRAPH

Basics of Graph

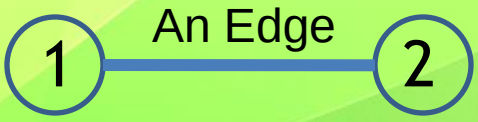
- A graph is a set of vertices(nodes) and edges.
- The node that holds data is called vertex and the line connecting two vertices is called edge.



A graph



A vertex



Basics of Graph

If ***G*** denotes a graph, ***G = (V, E)*** where ***V*** denotes set of vertices and ***E*** denotes set of edges.

$V = (v_1, v_2, v_3, \ldots, v_n)$ and $E = (e_1, e_2, e_3, \ldots, e_n)$,

where $e_i = (v_i, v_{i+1})$

Undirected Graph:

If each edge of a graph is undirected (without direction), the graph is called undirected graph.

The undirected edge is also called unordered edge.

Types of Graphe

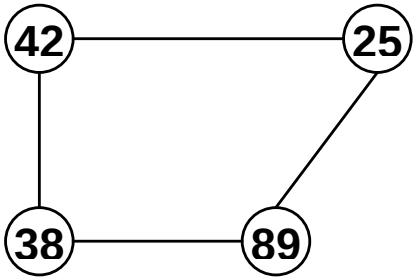
Directed Graph:

If each edge of a graph is directed (with direction), the graph is called directed graph.

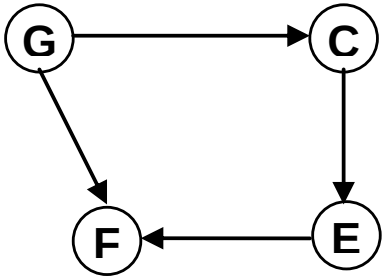
Weighted Graph:

If the value (cost) of each edge is given, the graph is called weighted graph.

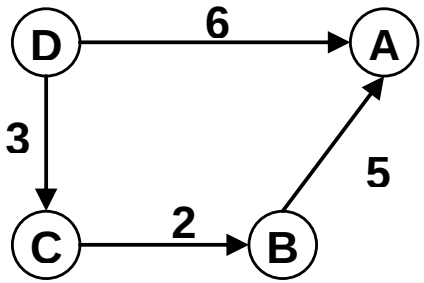
Basics of Graph [contd.]



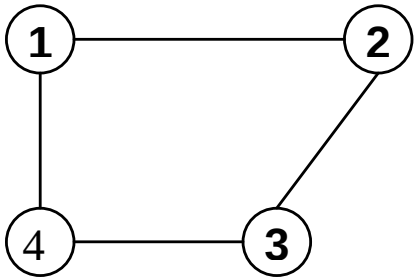
Undirected Graph



Directed Graph



Weighted Graph

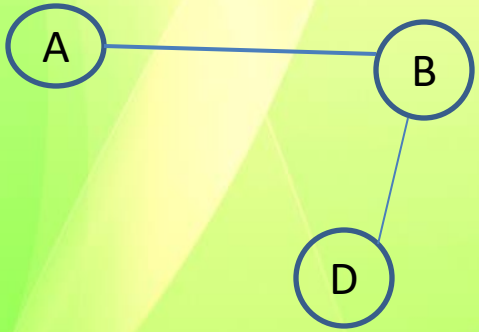


Graph

Figure 8.1: Different types of graphs

Basics of Graph [contd.]

Path: A path is a sequence of vertices where each pair of successive vertices is connected by an edge.



Cycle: A cycle is a path where first and last vertices are the same.



Graph Representation

- A graph is stored in the computer's memory using the matrix (two dimensional array)called adjacency matrix.
- Two vertices are adjacent to each other if there is an edge between them.
- If a vertex is adjacent to other, we put 1 in matrix on their cross point.
- If there is no edge, we put 0 in the matrix.

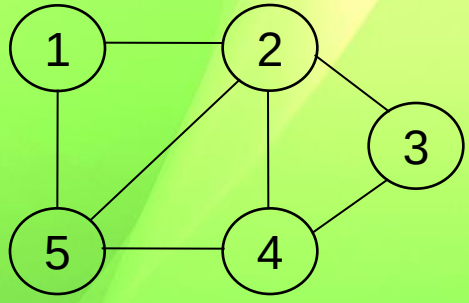
Graph Representation [contd.]

- $a_{ij} = 1$, if there is an edge between the vertices i and j. Otherwise, $a_{ij} = 0$, where A denotes adjacency matrix.

- In case of weighted graphs, we put edge value the adjacency matrix.

Graph Representation

- Adjacency matrix representation of G = (V, E)
  - Assume vertices are numbered 1, 2, … |V|
  - The representation consists of a matrix $A_{|V| \times |V|}$:
  - $a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$



Undirected graph

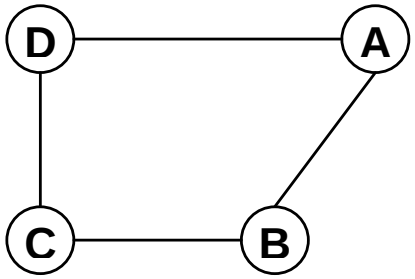
	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

For undirected graphs matrix A is symmetric:

$a_{ij} = a_{ji}$

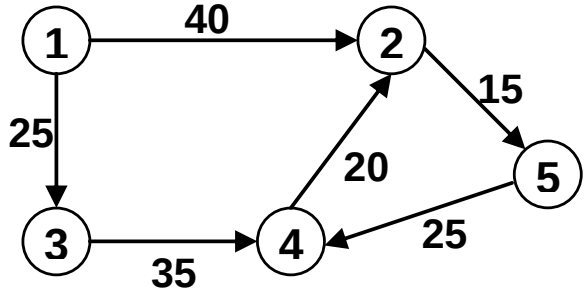
$A = A^T$

Graph Representation [contd.]



	A	B	C	D
A	0	1	0	1
B	1	0	1	0
C	0	1	0	1
D	1	0	1	0

(a): Adjacency matrix of an undirected graph

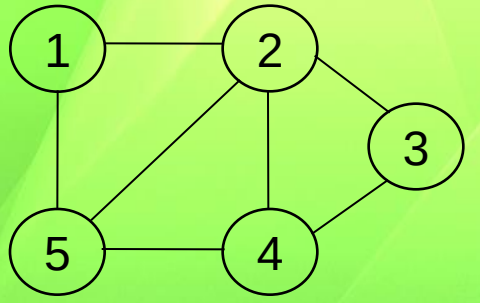


	1	2	3	4	5
1	0	40	25	0	0
2	0	0	0	0	15
3	0	0	0	35	0
4	0	20	0	0	0
5	0	0	0	25	0

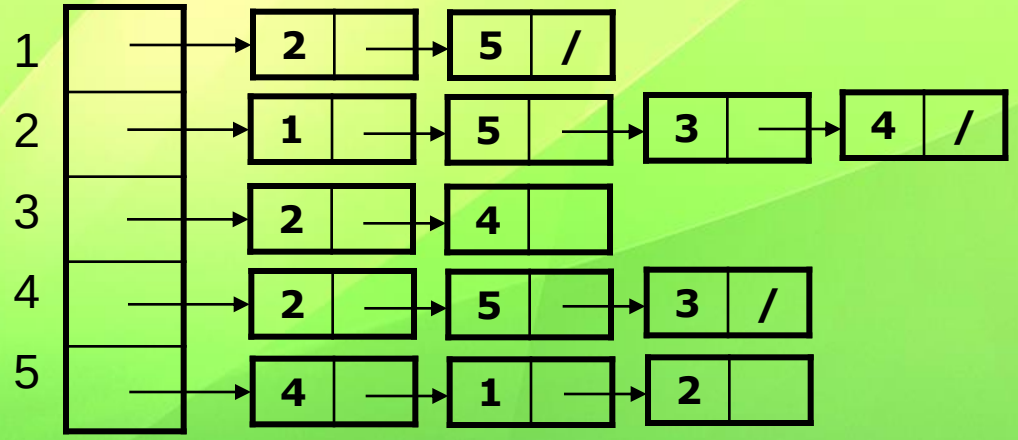
(b): Adjacency matrix of a directed and weighted graph

Graph Representation

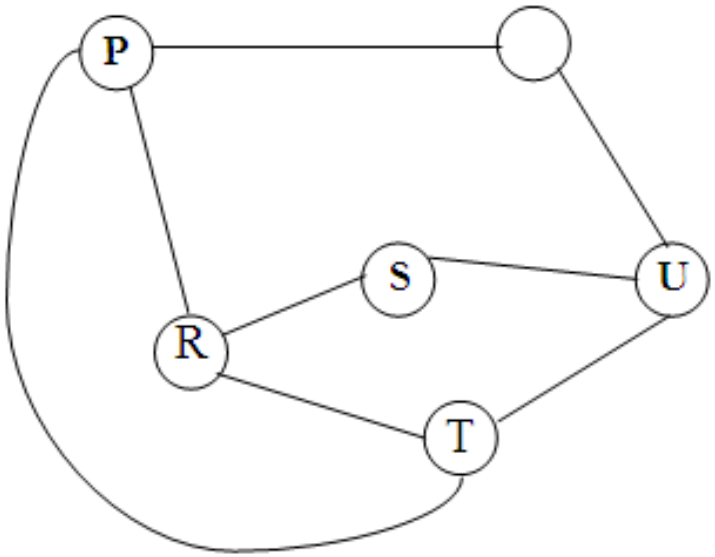
- Adjacency list representation of G = (V, E)
  - An array of |V| lists, one for each vertex in V
  - Each list *Adj*[u] contains all the vertices v such that there is an edge between u and v
    - *Adj*[u] contains the vertices adjacent to u (in arbitrary order)
  - Can be used for both directed and undirected graphs



Undirected graph



Graph Representation [contd.]



Vertices	List of adjacent vertices		
P	Q	R	T
Q	P	U	
R	P	S	T
S	U	R	
T	P	R	U
U	Q	S	T

Figure 8.3: An undirected graph and its adjacency list

Graph Representation [contd.]

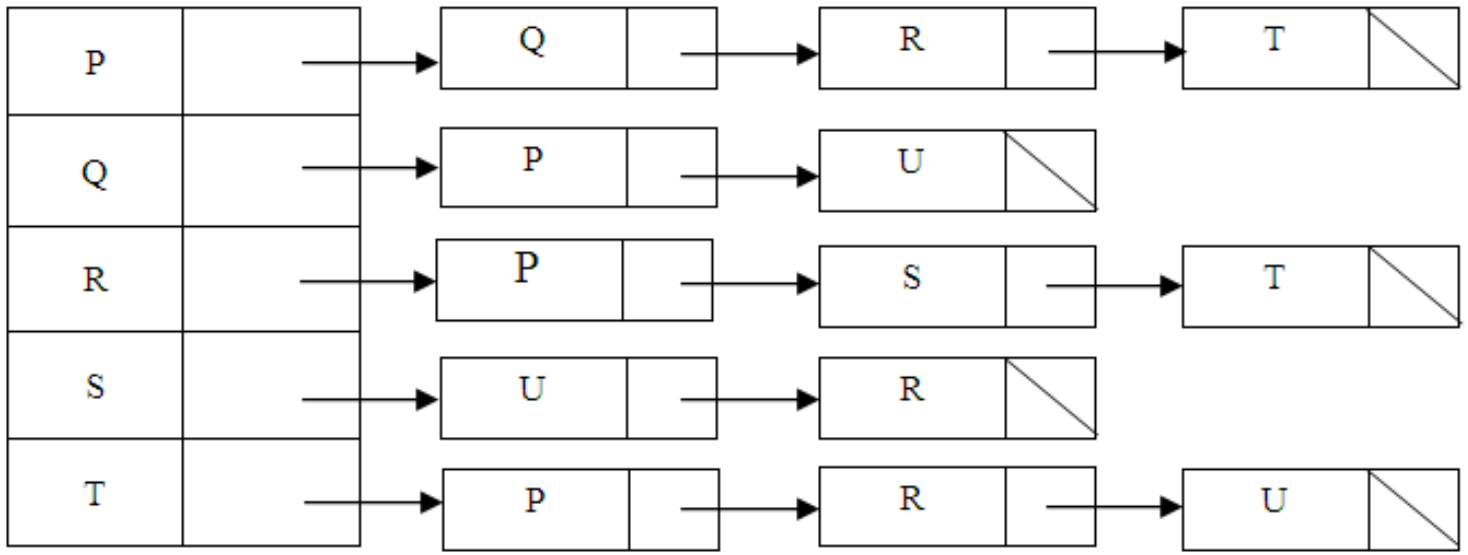


Figure 8.4: Linked representation of the adjacency lists

Algorithm to create adjacency list of a graph

1. Input a graph (information of graph) and take a variable, item.

2. Declare vertex (node) and table of vertices:

```
i) struct vertex
     {
     char data;
     vertex  * next;
   }
ii) vertex table [1,…, m], * nptr, * tptr;
```

3. Set i = 1

4. Create a table of vertices:

```
table [i].data = item;
table [i].next = NULL;
```

5. Create a vertex (node) with value:

```
nptr = new (vertex);
nptr → data = item;
nptr → next = NULL;
```

6. if (table[i].next = NULL), then

```
            table[i].next = nptr;
else {
tptr = table[i]. next
while (tptr → next ! = NULL)
{
tptr = tptr → next;
}
tptr → next = nptr;
```

7. Set i = i + 1;

8. Repeat step- 4 to 7 for necessary times

9. Output a linked adjacency lists.

Searching

Graph Traversal (Search) Methods

Graph Traversal Methods

- There are two principal methods for graph traversal, which are as follows:
  - Breadth First Search (BFS)
  - Depth First Search (DFS)

Graph Traversal Methods

- Example:

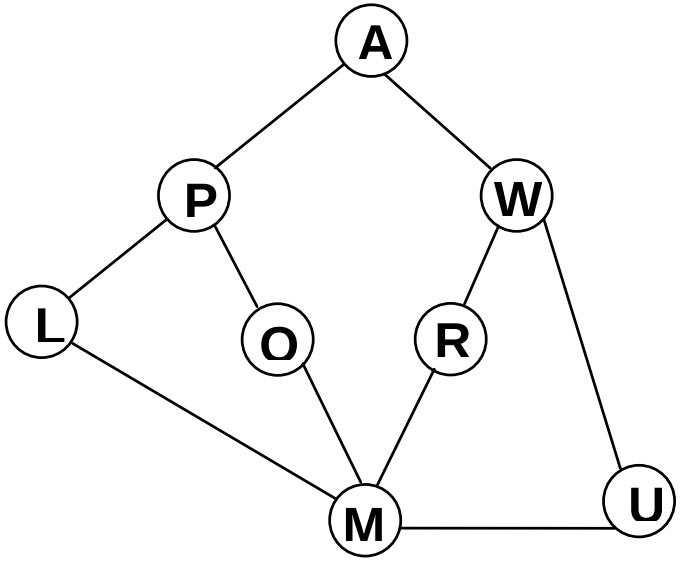


Figure 8.5: An undirected graph

Breadth First Search (BFS)

- In breadth-first search, we visit a vertex first and search for the vertices that are adjacent to this vertex. Take the adjacent vertices in a queue.
- The we visit (access) a vertex from queue and add the adjacent vertices of the vertex to queue.
- We shall continue this process until (the queue is empty) we visit every vertex in the graph.
- Remember a queue is FIFO stucture.

Breadth First Search (BFS) [contd.]

Table-8.1: Breadth first traversal of the graph in Figure-8.5

Vertices visited	Vertices in queue front	Vertex traversal done
A, W, P	W, P	A
A, W, P, U, R	P, U, R	W
A, W, P, U, R, O, L	U, R, O, L	P
A, W, P, U, R, O, L, M	R, O, L, M	U
	O, L, M	R
	L, M	O
	M	L
		M

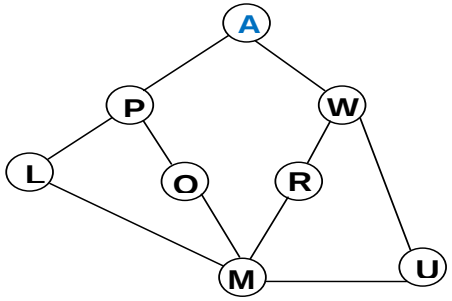


Figure 8.5: An undirected graph

Algorithm for breadth first search

1. Input a graph, G
2. Create two empty lists L and T
3. Create an empty queue Q
4. For each vertex of G do
    - i) Take a vertex *v* from G
    - ii) Place the vertex *v* and its adjacent vertices in L
    - iii) Add the adjacent vertices of the vertex *v* to Q
    - iv) Place the vertex *v* in T
5. Repeat the steps 6 to 7 until Q is not empty.
6. Access a vertex *w* from Q
7. For each adjacent vertex of *w*, do
    - i) If it (adjacent vertex of *w*) is not in L, place it in L and add it to Q
    - ii) Place vertex *w* in T
8. Output the traversal list in T

Depth First Search (DFS)

- In depth-first search, we visit a vertex and search for the vertices that are adjacent to the starting vertex. Add (push) the adjacent vertices to a stack.
- Then pop a vertex from the stack and push the adjacent vertices of the vertex to the stack.
- We shall continue the process until the stack is emply.
- Remember a stack is LIFO structure.

Depth First Search (DFS) [contd.]

Table 8.2: Depth first traversal of the graph in Figure-8.5

Vertices visited	Vertices in stack top	Vertex traversal done
A, P, W	P,W	A
A, P, W, R, U	P, R, U	W
A, P, W, R, U, M	P, R, M	U
A, P, W, R, U, M, L, O	P, R, L, O	M
	P, R, L	O
	P, R	L
	P	R
		P

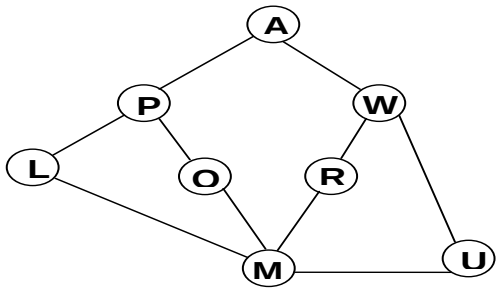


Figure 8.5: An undirected graph

Depth First Search (DFS) [contd.]

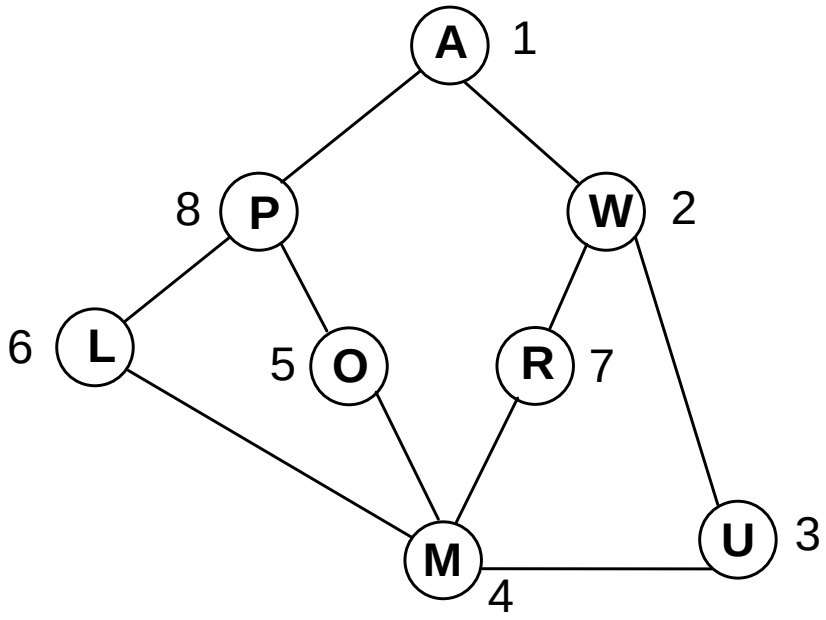


Figure 8.7: Pictorial view of depth first traversal method (traversal sequences are marked by 1, 2, 3, ….., 8)

Algorithm for depth first search

1. Input a graph, G
2. Create two empty lists L and T
3. Create an empty stack S
4. For each vertex of G do
   - i) Take a vertex *v* from G
   - ii) Place the vertex *v* and its adjacent vertices in L
   - iii) Add the adjacent vertices of the vertex *v* to S
   - iv) Place the vertex *v* in T
5. Repeat the steps 6 to 7 until S is not empty.
6. Access a vertex *w* from S
7. For each adjacent vertex of *w* do
   - i) If it (adjacent vertex of *w*) is not in L, place it in L and add it to S
   - ii) Place vertex *w* in T
8. Output the traversal list in T

Spanning tree Of a Graph

Minimum cost spanning tree

- **Spanning tree:**

  A spanning tree is a sub-graph of a graph that contains all the vertices and has no cycle.

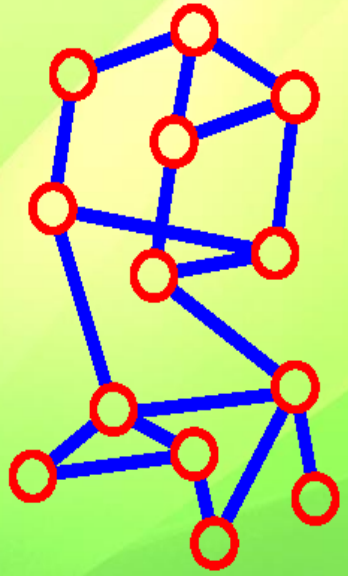
- **Minimum cost spanning tree:**

  It is a spanning tree whose cost is minimum. That is, the total weight of the edges will be minimum.

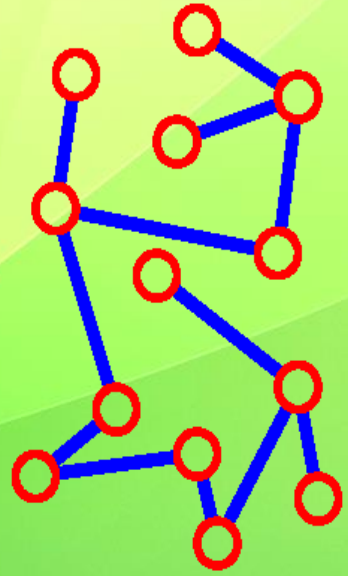
Spanning Tree

- A **spanning tree** of **G** is a subgraph which
  - is a tree
  - contains all vertices of **G**

- How many edges are there in a spanning tree, if there are *V* vertices?



G



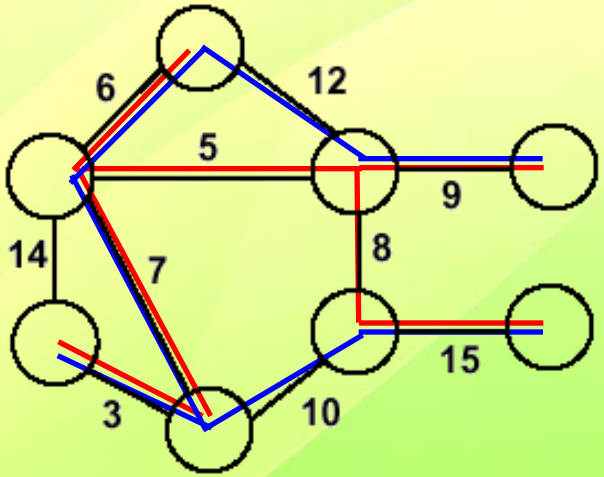
spanning tree of G

Minimum Spanning Trees

- Undirected, connected graph

  $G = (V, E)$

- **Weight** function $W: E \rightarrow \mathcal{R}$ (assigning cost or length or other values to edges)



- **Spanning tree**: tree that connects all vertices
- **Minimum spanning tree** (**MST**): spanning tree ***T*** that minimizes $w(T) = \sum_{(u,v)\in T} w(u,v)$

Algorithms

- We can construct minimum cost spanning tree using:

  1) Prim's algorithm

  2) Kruskal's algorithm

Example of Prim's algorithm

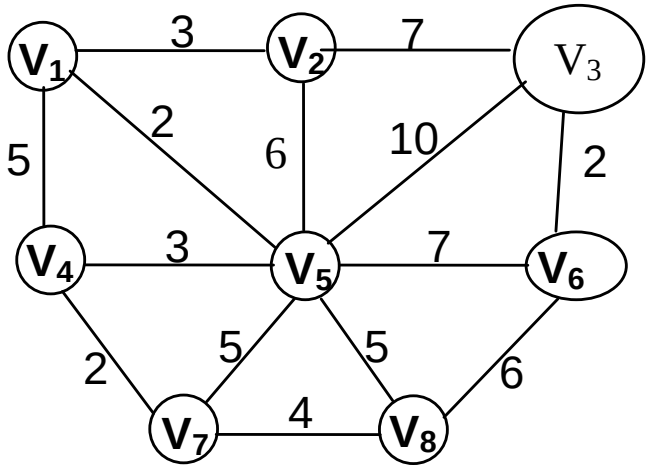
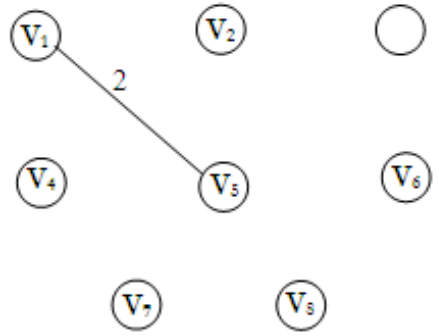
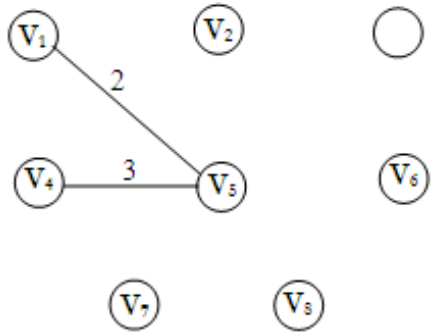


Figure 8.8: A weighted graph

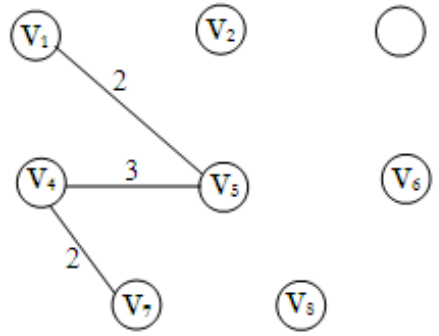
Prim's algorithm [contd.]



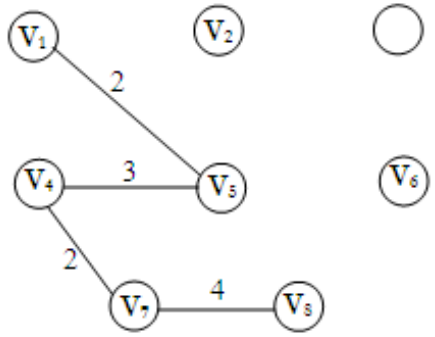
(a) Selection of first edge



(b) Selection of second edge

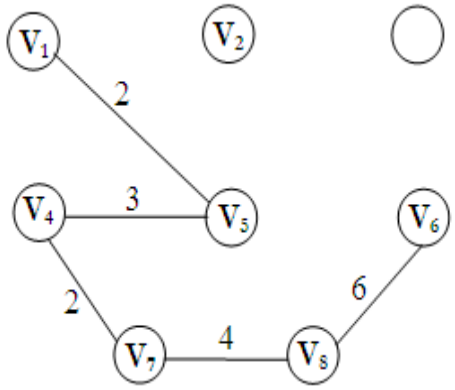


(c) Selection of third edge

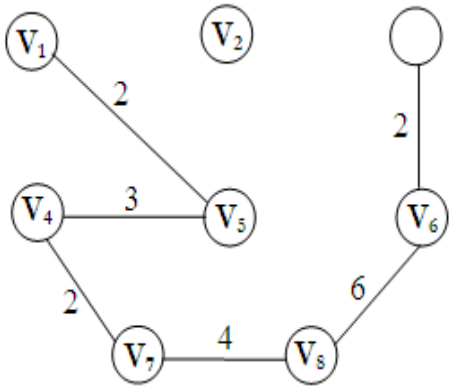


(d) Selection of fourth edge
Here we reject the
edge (V_7, V_5), since it makes a cycle

Prim's algorithm [contd.]

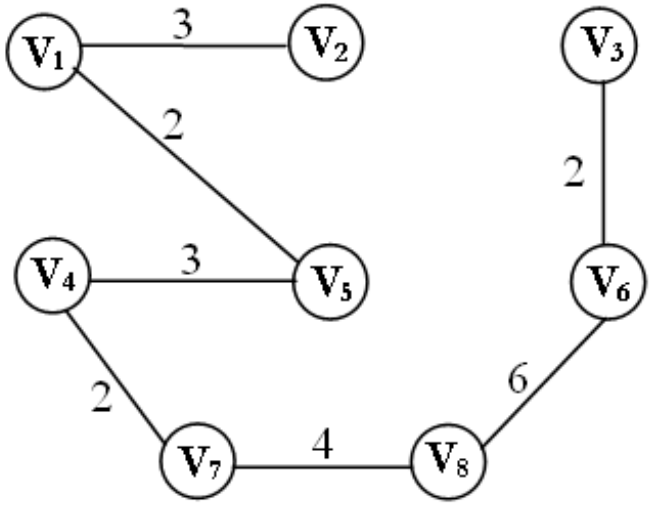


(e) Selection of fifth edge We cannot select the edge (V_8 , V_4), it has been already selected and we reject (V_8 , V_5), since it makes a cycle



(f) Selection of sixth edge

Prim's algorithm [contd.]



(g) Selection of seventh edge

Figure 8.9: Pictorial view of stages Prim's algorithm

Prim's algorithm [contd.]

- **Algorithm 8.4:** Prim's Algorithm

1. Input a weighted connected graph, G = (V, E). V is the set of vertices and E is the set of Edges
2. Create an empty list E_T
3. Create a list V_T which will initially contain the starting vertex, *i.e* $V_T = \{ v_0 \}$
4. for each vertex i of G do

   {

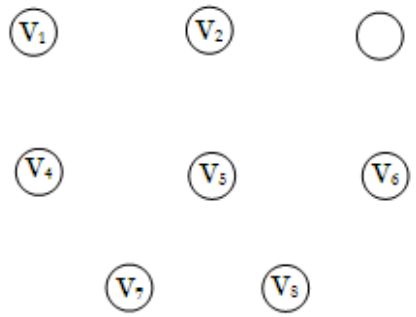
   find a minimum-weight edge e' = (v' , u') among all edges (v, u) such that v is in V_T and u is in $V - V_T$.

   $V_T = V_T \cup \{u'\}$;

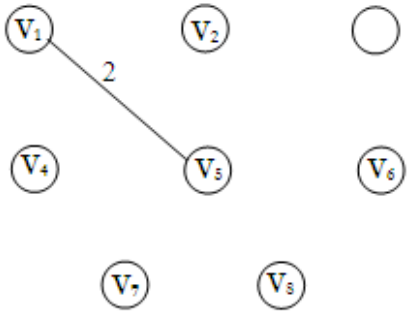
   $E_T = E_T \cup \{e'\}$;

   }
5. Output edge list of the spanning tree E_T.

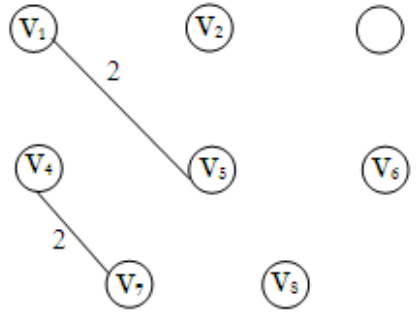
Example of Kruskal's algorithm



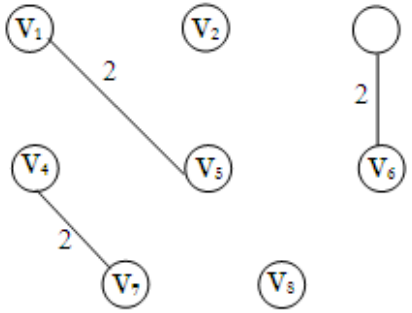
(a) Step 1 (Select all the nodes)



(b) Step 2 (Select an edge with minimum cost (minimum among all the edges))

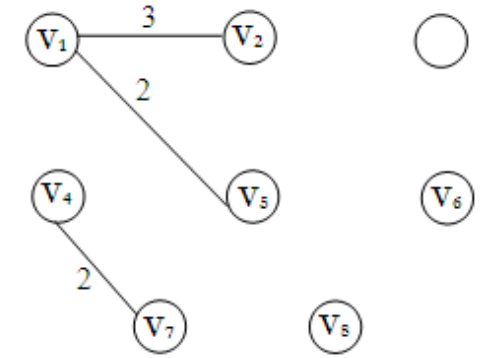


(c) Step 3

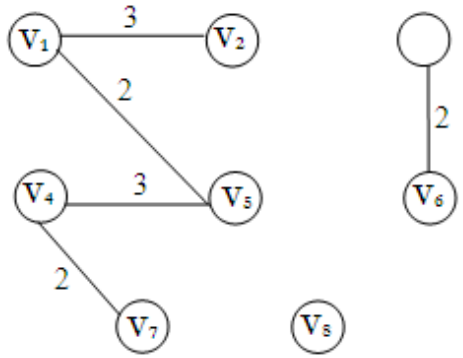


(d) Step 4

Kruskal's algorithm [contd.]

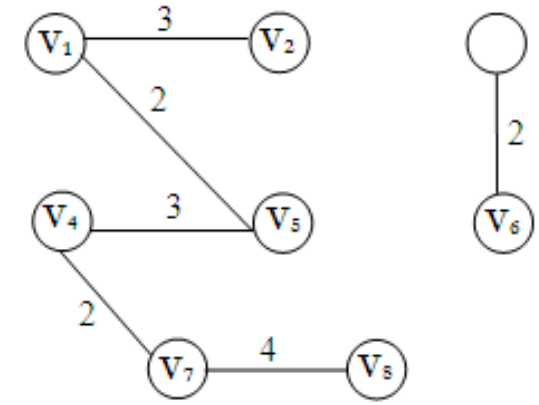


(e) Step 5

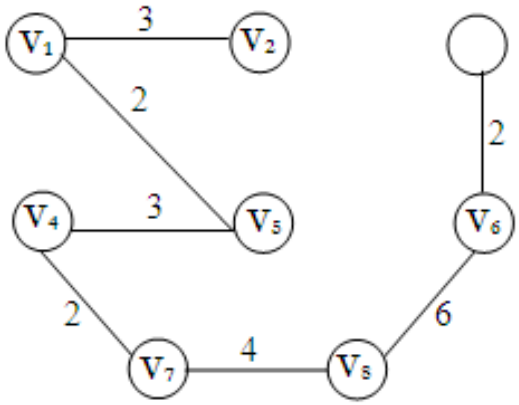


(f) Step 6

Kruskal's algorithm [contd.]



(g) Step 7



(h) Step 8

Figure 8.10: Pictorial view of the stages of the Kruskal's algorithm.

Kruskal's algorithm [contd.]

- **Algorithm 8.5:** Kruskal's Algorithm

1. Input a weighted connected graph, G = (V, E). V is the set of vertices and E is the set of Edges
2. Create an empty list E_T
3. Sort E in nondecreasing order of the edge weights $w(e_{i1}) \leq w(e_{i2}) \leq \ldots \leq w(e_{iE})$
4. Set ecounter = 0; k = 0;
5. while (ecounter < number of vertices - 1)

```
{
k = k + 1;
if (E_T U { e_ik } is acyclic)
E_T = E_T U { e_ik };
ecounter = ecounter +1 ;
}
```

6. Output edge list of the spanning tree E_T.

THANK YOU.